Ch#03 — Solving Problems By Searching

3.1 Introduction:-

A problem-solving agent is an agent that decides what to do by searching for a sequence of actions that will achieve a goal.

The previous chapter introduced agents and rational agents. This chapter focuses on how a goal-based agent finds the sequence of actions needed to reach its goal.

The basic idea is:

Initial State → Search → Goal Search

A search problem assumes an environment that is:

- Observable
- Deterministic
- Known
- Static

In such an environment, the agent can construct a sequence of actions before executing them.

Scanned with CamScanner

Problem-Solving Agent

A problem-solving agent works through the following stages:

Perceive → Goal Formulation → Problem Formulation → Search → Execute.

~~Goal Formulation~~:-

1. Problem-Solving by Searching:-

A problem-solving agent is a type of goal-based agent that finds a sequence of actions that leads from the current state to a desired goal state.

A single action is often not enough to achieve a goal. Therefore, the agent must determine a sequence of actions.

Example: A person wants to travel from Karachi to Lahore.

The agent must determine a sequence such as:

Karachi → Hyderabad → Sukkar → Multan → Lahore

The process of finding such a sequence is called ~~searching~~ search.

2. Problem-Solving Agent :-

A problem-solving agent :

1. Perceives the current situation
2. Formulates a goal
3. Formulates the problem
4. Searches for a solution.
5. Execute the solution.

The problem-solving agent uses atomic representation, meaning that states are treated as complete units without considering their internal structure.

Goal Formulation :-

It is the process of deciding what the agent wants to achieve. The goal based on:

- Current situation
- Agent's performance measure
- Desired outcomes

A goal is a set of world states in which the desired condition is satisfied.

- Example: If a robot is at room A and wants to reach room D.
- Current state: Room A
- Goal: Reach Room D

Scanned with CamScanner

The agent does not initially need to know exactly it will reach Room D.

The goal must be identified before the problem is formulated because the goal determines :

- Which states are relevant
- Which actions need to be considered
- What counts as a solution

Problem Formulation :-

It is the process of deciding which states and actions should be considered in order to achieve a given goal.

After deciding the goal, the agent describes the problem in a form that a search algorithm can solve.

A well - defined problem contains 5 components :

1. Initial state
2. Actions
3. Transition model / successor function
4. Goal test
5. Path cost

i) Initial State :-

It is the state in which the agent starts.

Scanned with CamScanner

- Example: Water-Pouring Problem

There is a 4 and 3 gallon bucket. Initially both buckets are empty.

The initial state is: (0,0)

ii) Actions:-

Actions are the operations that the agent can perform in a particular state.

- Examples in water-pouring problem:

- Fill a bucket
- Empty a bucket
- Pour one bucket into another.

The available actions determine how the agent can move from one state to another.

iii) Transition Model:-

The transition model describes what happens when an action is performed. It tell us: If the agent is in state S and performs action A, what state will result?

It can be represented as: Result (s, a) = s'

S = current state a = action
S' = resulting state

iii) Successor Function :-

It gives all possible states that can be reached from a given state by applying the available actions.

It produces : Action → Resulting state

- Example : From state (0,0), possible successors include :

$(0,0) \rightarrow (3,0)$
$(0,0) \rightarrow (0,4)$

The successor function is therefore used to generate the possible next states during search.

iv) Goal ~~State~~ Test :-

A goal test determines whether a given state satisfies the goal. It returns true and false accordingly.

- Water - pouring example : The goal is to have exactly 2 gallons in one bucket.

Therefore :

$(x, 2)$ or $(2, x)$ is a goal state.

v) Path Cost :-

The path cost is the total cost of the sequence of actions used to reach a state. It is usually represented by : $g(n)$

Scanned with CamScanner

Goal first → Problem second → Search third → execute last

where $g(n)$ is the cost of the path from the initial state to node n.

* A solution with lower path cost is cheaper.

State :-

A state represents a particular configuration of the problem. It describes what the world is like at a particular point.

Example : Water-pouring : (2, 3)

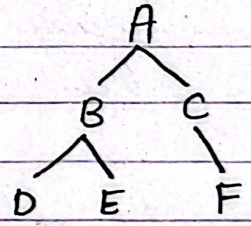
means :

- 2 gallons in one bucket
- 3 gallons in the other bucket.

State Space :-

The state space is the set of all possible states that can be reached from the initial state by applying valid actions.

Example :



The state space consists of : {A, B, C, D, E, F}

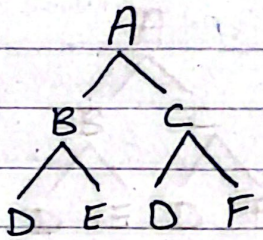
* A search algorithm explores this state space to find a path from the initial state to a goal state.

Scanned with CamScanner

Search Tree :-

It represents the different paths that the search algorithm can explore.
The root represents the initial state.
Each child represents a possible action.

Example :-



The state D is the same state, but it appears through two different paths.

The same state can appear multiple times in a search tree if it can be reached through different paths.

Search Node :-

A node is a data structure used by the search algorithm to represent a state in the search tree.

A node normally contains :

- State
- Parent
- Action
- Path cost
- Depth

Scanned with CamScanner

Node Structure:

Node = (State, Parent, Action, PathCost, Depth)

State vs Node:

State describes a situation in the problem.

Node is a search algorithm's record containing that state and information about how it was reached.

Search :-

Search is the process of exploring possible states and action sequences to find a path from the initial state to a goal state.

The search algorithm repeatedly:

1. Selects a node
2. Expands the node
3. Generates successors nodes.
4. Tests whether a goal has been reached.

Solution :-

A solution is a sequence of actions that leads from a initial state to a goal state.

Scanned with CamScanner

Optimal Solution :-

An optimal solution is a solution with the lowest possible path cost. If several solutions exists : S_1 : cost = 10, S_2 : cost = 20 then S_1 is the optimal solution.

Abstraction :-

Real-world problems contain enormous amounts of unnecessary detail.

Abstraction removes irrelevant details while keeping the information necessary for solving the problem.

A good abstraction reduces the size of the problem and makes search more efficient.

Node Expansion :-

To expand a node means to generate its successors by applying all applicable actions.

The function that generates successors nodes is called the EXPAND operation.

Tree Search & Graph Search..

There are two general ways to perform search.

Tree Search :-

Tree search treats different paths as separate nodes and does not automatically remember previously explored states.

This can cause the same state to be explored repeatedly.

- Problem : Repeated states can waste large amount of time and memory.

Graph Search :-

Graph search keep track of previously explored states. If a state has already been explored, the algorithm can avoid exploring it again.

Main Advantage :-

Graph search helps prevent :

- Repeated states
- Cycles
- Redundant paths

Scanned with CamScanner

Repeated states :-

It occurs when the same state is reached more than once during search. Repeated states commonly occur when actions are reversible.

Example :-

- Route finding
- Sliding - block puzzles
- Moving between connected locations.

Failure to detect repeated states can make a search extremely ineffecient.

Problem Environment Assumptions:

The environment is:

- Static: The world does not change while the agent is planning.
- Observable: The agent knows the relevant state.
- Discrete: States and actions can be explicitly represented/enumerated.
- Deterministic: An action produces a predictable result

Therefore, the agent can plan a sequence of actions in advance and execute it.
This is an open-loop system.

Scanned with CamScanner

Complexity of Search :-

The complexity of search problem describes the resources required to find a solution.

The main resources are:

- **Time**: how many nodes are generated/expanded
- **Space**: how much memory is required.

The 2 important parameters are:

Branching Factor b :-

The branching factor is the number of successors generated from a node.

Example: If each state has approximately 3 possible actions : $b = 3$

Depth d :-

The depth of the shallowest solution is the number of actions from the initial state to the nearest goal.

Maximum Depth m :-

It represents the maximum depth of the search tree.

* This is particularly relevant to DFS.

Scanned with CamScanner

Evaluation Criteria of Search Algorithms :-

Search algorithms are evaluated using four main criteria.

a) Completeness :-

A search algorithm is complete if it is guaranteed to find a solution whenever a solution exists.

b) Optimality :-

It is optimal if it is guaranteed to find the least-cost solution.

c) Time Complexity :-

Measures how much computational work the algorithm performs, usually in terms of the number of nodes generated.

d) Space Complexity :-

Measures how much memory the algorithm requires during the search.

Search Strategies:-

After a problem has been formulated, the agent must search the state space to find a solution.

A search strategy determines which node should be expanded next.
Search Algorithms are divided into:

1. Uninformed Search.
2. Informed Search.

Uninformed Search:-

They are also called blind search, uses only the information contained in the problem definition.

It has no additional knowledge about where the goal is and does not use a heuristic.

Main Uninformed Search Algorithms:-

1. Breadth-First Search (BFS)
2. Uniform-Cost Search (UCS)
3. Depth-First Search (DFS)
4. Depth-Limited Search (DLS)
5. Iterative Deepening Search (IDS)

Scanned with CamScanner

- Greedy Best-First Search
- A* Search

Informed Search :-

Informed search uses additional information about the problem to determine which states are more promising.

The additional information is usually provided by a heuristic function.

$$h(n)$$

where $h(n)$ estimates the cost of reaching a goal from node n.

Heuristic Function :-

A heuristic function is an estimate of the remaining cost from the current node to a goal.

$h(n)$ = estimated cost from n to a goal

It helps the search decide where to look first.

- Example: In a route finding, the straight-line distance from the current city to the destination can be used as a heuristic.

If : $h(A) = 500$ km

then the estimated remaining distance from A to the goal is 500 km.

Scanned with CamScanner